

Allowing Oscillation Quantization: Overcoming Solution Space Limitation in Low Bit-Width Quantization

Weiyang Xie¹ Zihan Meng^{1*} Jitao Ma¹ Wenjin Guo¹ Haowei Li¹
 Haonan Qin¹ Leyuan Fang² Yunsong Li¹
¹Xidian University ²Hunan University

Abstract

Quantization-aware Training (QAT) enables deep models to adapt to precision loss by simulating quantization. However, existing methods often converge to sub-optimal solutions due to inadequate exploration of quantization solution space. To address this, we propose a novel QAT method, Allowing Oscillation Quantization (AOQ), which expands the reachable solution space through weight oscillation. Notably, unlike previous methods that suppress oscillation throughout training, AOQ actively encourages it in the earlier stages to explore diverse quantization configurations, and suppresses it later to ensure convergence. In addition, by decoupling quantization thresholds and levels, AOQ promotes meaningful oscillation and improves the stability of learnable quantization parameters. Extensive experiments across various models, including ResNet, MobileNet, DeiT and Swin Transformer, demonstrate the effectiveness of our method. Specifically, with 2-bit quantization, AOQ achieves a 0.4% ~ 2.2% accuracy improvement on ImageNet compared to state-of-the-art methods. Our implementation is available at <https://github.com/muzenc/AOQ>.

1. Introduction

Model quantization [31, 46, 49] has attracted growing attention for enabling efficient model inference while substantially reducing hardware resource consumption. By reducing the bit-width of model parameters, quantization effectively reduces memory footprint and computational complexity, thereby enhancing inference speed and energy efficiency. Thus, quantization has been widely applied in tasks such as image classification [27, 32, 39], object detection [3, 42, 48] and natural language processing [1, 10, 11].

Despite its advantages, quantization shrinks the dynamic range of parameters, often resulting in information loss and degraded performance. Quantization-aware training (QAT) significantly mitigates the impact of quantization on perfor-

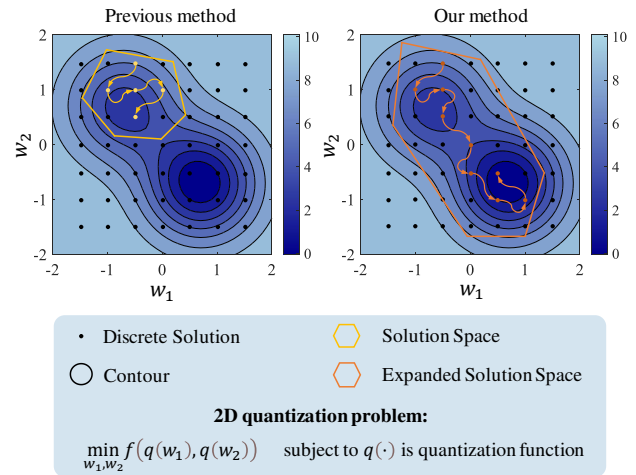


Figure 1. Illustration of solving a 2D quantization problem using previous methods vs. proposed method. The two plots are contour maps of the function $f(w_1, w_2)$, where darker regions indicate lower function value. The yellow and orange curves show the solution trajectories of the previous method and our method, respectively. In contrast, AOQ explores a broader solution space and finds a better solution.

mance compared to post-training quantization [16, 24, 35], but a noticeable performance gap remains under extremely low-bit quantization scenarios, such as 2-bit quantization.

Mainstream QAT techniques [12, 32, 46] simulate quantization effects by inserting fake quantization nodes during training, allowing the model to adapt to low-precision constraints. However, these nodes are non-differentiable, leading prior works [13, 23, 45] to employ biased gradient approximations. Recent studies [8, 30, 36] show that such biased gradients can result in weight oscillation, hindering the network’s convergence to a better local minimum. Nevertheless, an overlooked insight is that QAT relies on weight oscillation to explore more quantized solutions. In existing methods [12, 13, 18, 32], the frequency of weight oscillation is relatively low, and directly suppressing oscillations may limit the model’s ability to escape suboptimal quanti-

*Corresponding author.

zation bins. Specifically, weights seldom cross quantization thresholds, instead making insignificant shifts within quantization bins.

We investigate QAT from the perspective of solving non-linear mixed-integer programming problems, drawing an analogy to the branch-and-bound method with depth-first search (DFS). While DFS systematically explores all possible solutions through branching and backtracking, QAT lacks an exhaustive exploration mechanism and instead performs local updates guided by gradients. Our analysis reveals that in existing methods, most weights infrequently change their quantization bin assignments, thereby restricting the search to a small subset of possible quantization solutions and resulting in a limited solution space. As shown in Figure 1, this limited search space often leads to premature convergence, leaving potentially optimal configurations unexamined. According to [37], low bit-width quantization introduces sharp curvature in the loss landscape, indirectly demonstrating that gradient-based optimization is more likely to get trapped in poor local minima.

To address the problems mentioned above, we propose a novel QAT approach, named Allowing Oscillation Quantization (AOQ). Unlike previous methods that suppress weight oscillation throughout the entire optimization process, AOQ strategically leverages weight oscillation as a positive exploration mechanism. We encourage weight oscillation to expand the solution space of the weights and only suppress oscillation in the later stages of training. Moreover, AOQ addresses a key limitation of traditional QAT methods, where the coupling of quantization thresholds and levels often causes inaccurate updates to the quantization parameters and hinders model convergence [30]. To overcome this, AOQ decouples thresholds and levels, allowing thresholds to be manually adjusted to encourage oscillation while setting quantization levels as learnable parameters directly optimized with the task loss. Our contributions can be summarized as follows:

- We empirically show that fake quantization-based QAT methods explore a limited subset of quantized solutions and are prone to suboptimal convergence. To the best of our knowledge, this is the first systematic demonstration of this phenomenon.
- We propose AOQ to expand the quantization solution space of the weights by leveraging weight oscillation. Moreover, we improve the stability of learnable quantization parameters by decoupling the thresholds and levels.
- Extensive experiments in CNN and Vision Transformer architectures demonstrate the effectiveness of AOQ in extremely low-bit quantization.

2. Related Work

QAT was initially proposed for binary quantization [7], and has since become the most popular quantization method in

precision-sensitive scenarios. Most QAT-based methods insert fake quantization nodes into the model. These nodes simulate the effects of quantization during forward propagation and pass the approximated gradients during backward propagation [18, 19, 26, 27, 43, 46]. Some other methods [8, 38] add independent pseudo quantization noise during training, enabling effective QAT without relying on gradient approximations. This work focuses on the former.

The former is non-differentiable at the fake quantization nodes, preventing optimization and convergence of the quantized model. To address this problem, earlier works approximate gradients using straight-through estimator (STE) [22, 45] and its variants [13, 23, 44], while other approaches compute gradients using constructed neural networks or manual derivation [4, 25]. Among them, STE is widely used in quantization methods due to its robustness, broad applicability, and straightforward implementation. However, it inevitably brings the problem of gradient mismatch: gradients of weights are computed based on their quantized values, instead of their floating-point values. Previous research demonstrated that following the STE approach can cause weight oscillation during training due to biased gradients [8]. It has been suggested that weight oscillation prevents the network from converging to the best local minimum, and that suppressing oscillation earlier is beneficial for the optimization process [36]. The *oscillation dampening* and *iterative weight freezing* methods it proposed overcome weight oscillation. In addition, OFQ [30] observed that weight oscillation can introduce noise to the gradient of the learnable scaling factor (quantization level) and cause it to oscillate. The latter, in turn, changes the quantization threshold and aggravates the weight oscillation. OFQ deems weight oscillation harmful and suppresses it; in contrast, AOQ dispels this long-standing misconception, theoretically and experimentally proving its value.

Moreover, several works have delved into searching for optimal scaling factor to determine quantization thresholds and levels. For instance, trainable clipping parameters have been introduced to determine the appropriate quantization scale [5, 49]. Esser *et al.* [12] provided a simple yet effective way to approximate the gradient of learnable scaling factor, which is sensitive to quantized state transitions. It has been widely adopted as the *de facto* approach for QAT. Another line of research decouples quantization thresholds and levels, proposing increasing the quantization representation ability by replacing uniform quantization with non-uniform quantization [32]. The approach has demonstrated excellent performance in quantization of activation values.

While we acknowledge that weight oscillation impacts final convergence, we argue that it is not entirely detrimental to optimization. In layer-by-layer quantization, each weight represents a dimension within the quantization solution space, and weight oscillation plays a crucial role in

rapidly exploring optimal quantization solutions within limited training iterations. Moreover, our research shows that in existing methods, the oscillation frequency of weights at quantization thresholds is quite low, motivating us to increase this frequency. To this end, we delve into the working mechanism of QAT, and our proposed AOQ method combines the benefits of enhancing weight oscillation to expand the solution space and suppressing weight oscillation to ensure stable convergence.

3. Methodology

3.1. Preliminaries

One of the most effective ways to quantize a neural network is by inserting fake quantization nodes to simulate the quantization effect during training. In weight quantization, the goal is to identify the quantization bin each weight falls into and assign it the corresponding quantization level. At present, the most successful methods are based on LSQ [12], and its quantization function is:

$$\hat{w} = q(w; s, Q_n, Q_p) = s \cdot \text{clip}\left(\left\lfloor \frac{w}{s} \right\rfloor, Q_n, Q_p\right), \quad (1)$$

where $\text{clip}(\cdot, \alpha, \beta)$ is a truncated function with lower bound α and upper bound β , $\lfloor \cdot \rfloor$ is a rounding operator, s is a scaling factor, Q_n and Q_p are the quantization ranges.

Since the quantization function is non-differentiable, the gradient of w cannot be directly computed during backpropagation. Thus, gradient approximation is used to keep the training going, such as straight through estimator (STE) [2]:

$$\frac{\partial L}{\partial w} \approx \frac{\partial L}{\partial \hat{w}} \cdot \mathbf{1}_{Q_n \leq w/s \leq Q_p}, \quad (2)$$

where $\mathbf{1}$ is an indicator function. When $Q_n \leq w/s \leq Q_p$, it is 1, otherwise it is 0. There is no gradient for w outside the quantization range. In LSQ, the gradient of the scaling factor s is computed as:

$$\begin{aligned} \frac{\partial L}{\partial s} &= \frac{\partial L}{\partial \hat{w}} \cdot \frac{\partial \hat{w}}{\partial s}, \\ \frac{\partial \hat{w}}{\partial s} &= \begin{cases} -\frac{w}{s} + \left\lfloor \frac{w}{s} \right\rfloor & \text{if } Q_n < \frac{w}{s} < Q_p \\ Q_n & \text{if } \frac{w}{s} \leq Q_n \\ Q_p & \text{if } \frac{w}{s} \geq Q_p \end{cases}. \end{aligned} \quad (3)$$

3.2. Limited Solution Space of Weights

Neural network quantization can be formulated as a mixed-integer nonlinear programming (MINLP) problem. It involves converting floating-point weights and activations into a finite set of discrete values while minimizing performance degradation. In this formulation, quantized weights and activations act as integer variables, while the nonlinearity stems from forward propagation and activation functions. Continuous variables, such as quantization parameters, are also involved. The optimization objective is:

$$\min_{\mathbf{w}} \mathbb{E}_{(x,y) \sim D} [\mathcal{L}(f_{\text{quant}}(x; q(\mathbf{w})), y)], \quad (4)$$

where $\mathcal{L}(\cdot)$ is the loss function, $\mathbf{w}$ is the set of all weights, $(x, y) \sim D$ represents samples drawn from the dataset D , $f_{\text{quant}}(\cdot)$ represents the model's output with activation quantization, and $q(\cdot)$ is the quantization function. $q(\mathbf{w})$ consists of integer variables $\mathbf{Q}$ and continuous parameters s, z :

$$\forall \mathbf{Q} \in [0, 2^b - 1], \mathbf{Q} \in \mathbb{Z}, b \in \mathbb{N}_*, q(\mathbf{w}) = s \cdot (\mathbf{Q} - z), \quad (5)$$

where $\mathbf{Q}$ is the set of integers mapped from $\mathbf{w}$, s is the scaling factor, z is the zero point, and b is the bit-width.

QAT-based methods serve as heuristics for solving this MINLP problem. These methods select an initial solution and iteratively refine it. Drawing inspiration from research on mixed-integer programming [6, 47], we observe that QAT shares similarities with depth-first search (DFS) in branch-and-bound algorithms. Both rely on branching and backtracking: QAT chooses quantization strategies per weight (branching), and uses gradient updates (implicit backtracking) to adjust solutions. However, unlike exact DFS, which systematically explores the entire solution space, QAT heuristics prioritize fast convergence, often leading to limited exploration and suboptimal results. We compare QAT and DFS from two key perspectives:

- **Branching of variables** In DFS, the algorithm explicitly branches on relaxed variables to construct the search tree. Likewise, in QAT, weights oscillating near quantization thresholds effectively induce branching behavior, determining whether a weight falls into one bin or another.
- **Backtracking mechanism** DFS performs explicit backtracking: when a branch leads to no improvement, it reverts and explores alternatives, enabling a global search. In contrast, QAT relies on local gradients to implicitly adjust weights. This often causes the model to converge prematurely and miss better configurations.

In summary, both DFS and QAT can be regarded as rule-based search strategies. However, under low-bit constraints, QAT's gradient-based updates are more prone to becoming trapped in local optima due to the steep curvature of the loss surface [37]. Motivated by this limitation, we aim to enhance QAT by expanding its effective solution space, similar in spirit to the global search behavior exhibited by exact DFS, in order to achieve improved performance.

We assess the ability of existing methods to explore the weight solution space by measuring how often weights cross quantization thresholds. As shown in Table 1, we use LSQ [12] to quantize ResNet-18 on the ImageNet [9]. "SGD" denotes an SGD optimizer with an initial learning rate of $1e-2$ and cosine learning rate decay [34], while "Adam" denotes an Adam optimizer with a learning rate of $1e-3$, both following the training strategies in [12]. We find that the frequency of weights crossing quantization thresholds is very low, all below 0.5%, with most untruncated weights concentrated in the $[0, 0.01\%)$ range. This indicates that previous gradient-based optimization methods have a

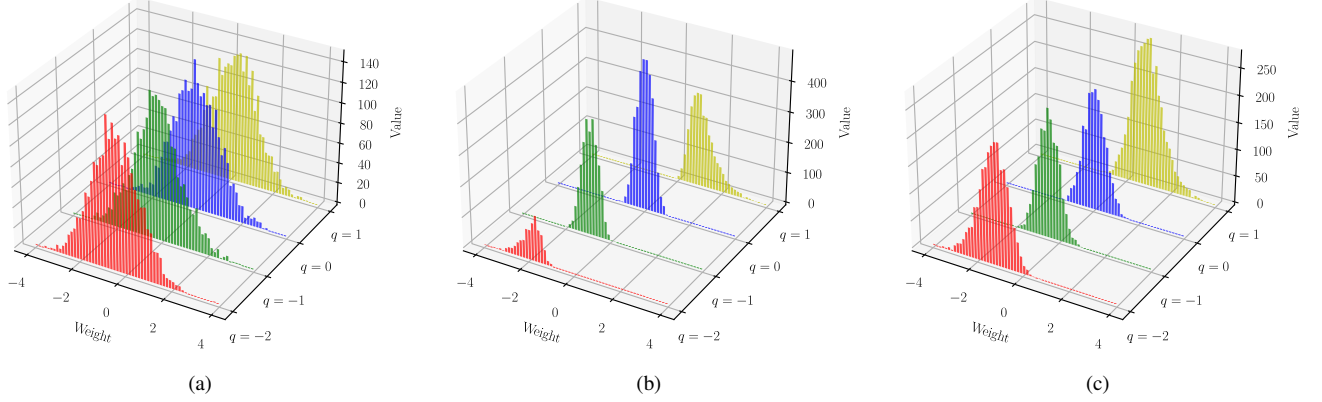


Figure 2. Histograms of pre-trained weights mapped to different quantization values: (a) lossless solution; (b) LSQ; and (c) our AOQ. The red, green, blue, and yellow bars represent histograms of pre-trained weights mapped to the quantized values -2, -1, 0, and 1, respectively.

Table 1. Measuring the solution space of weights on 2-bit quantized ResNet-18. The first column is the frequency of weights crossing the quantization thresholds, calculated as the number of crossings divided by the total iterations. “Truncated” refers to weights whose frequency of crossing quantization thresholds falls between 0 and 0.01% due to being clipped by the truncation function. “Not Truncated” refers to not being clipped by the truncation function. “% of weights” refers to the percentage of weights within a specific frequency range.

Frequency Range	% of weights	
	SGD	Adam
[0, 0.01%) - Truncated	14.5%	11.8%
[0, 0.01%) - Not Truncated	47.2%	12.6%
[0.01%, 0.05%)	5.1%	19.1%
[0.05%, 0.1%)	8.4%	24.5%
[0.1%, 0.5%)	24.8%	32.0%

weak ability to explore the solution space, potentially hindering the model from finding an optimal solution.

To further illustrate how limited exploration affects quantization quality, we construct a toy example. We first generate an integer set $\bar{w} \in \mathbb{R}^{10000}$ containing only values -2, -1, 0, and 1. We then derive matrix $\mathbf{A} \in \mathbb{R}^{2000 \times 10000}$ and vector $\mathbf{b} \in \mathbb{R}^{2000}$ satisfying $\mathbf{A}\bar{w} = \mathbf{b}$, establishing $\bar{w}$ as our known lossless integer solution. Clearly, $\mathbf{A}$ is not a full-rank matrix, which indicates that the linear equation system has multiple solutions, similar to the multi-solution nature of neural networks. Solving $\mathbf{A}w = \mathbf{b}$ with the constraint $w \in \{-2s + z, -s + z, z, s + z\}^{10000}$, where $s, z \in \mathbb{R}$, simulates the quantization process. Conceptually, this setup mimics training a fully connected network with 2-bit quantization, 10,000 input units, one output unit, and 2,000 samples, using mean squared error loss and Adam optimizer with a learning rate of 1×10^{-3} .

We first train the model with floating-point weights until convergence, obtaining pre-trained weights, similar to the standard QAT procedure. These weights are then mapped to quantized values. In the lossless solution, the scaling factor s is 1 and z is 0. Figure 2a shows the distribution of pre-trained weights mapped to each quantized value under the lossless solution. Each quantization level corresponds to a wide range of pre-trained weights, indicating that allowing distant weights to map to the same quantized value can preserve model quality. In contrast, Figure 2b shows the distribution after freezing s to 1 and z to 0 and using LSQ [12] for quantization. In this setting, pre-trained weights are mapped only to the nearest quantized values, restricting the solution space. Such methods, by mapping only to nearby values, may result in more mapping errors and does not fully leverage the broader solution space to minimize quantization errors.

3.3. Allowing Oscillation Quantization

In previous methods, many weights seldom oscillate near quantization thresholds, limiting their ability to switch quantization bins and often resulting in suboptimal solutions. We speculate that in QAT, especially in low-bit settings, weight transitions between different quantization levels make gradient direction unstable. This instability causes weights to fluctuate within a narrow range during training, preventing them from crossing thresholds. While some studies [30, 36] consider weight oscillation harmful to the whole optimization process, we believe that oscillation actually promotes solution diversity, increasing the likelihood of discovering better optima. Specifically, oscillation acts as an adaptive perturbation mechanism that enables the model to continuously test new quantization configurations and explore more quantized solutions within a given number of iterations, helping avoid premature convergence. This phenomenon resembles “random perturbation” strategies used

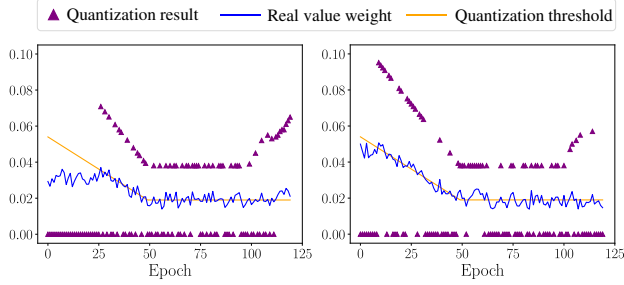


Figure 3. Oscillation patterns of two weights in layer1.0.conv1 of ResNet-18 during early training. We apply AOQ under a 2-bit quantization setting and monitor quantization assignments at the end of each epoch.

in integer programming [21, 40], where moderate instability prevents the model from becoming locked into overly deterministic paths. Based on this intuition, we propose to actively promote oscillation during early training phases to encourage exploration, then gradually reduce it in later stages to ensure convergence stability.

To increase the number of weights that cross thresholds without disrupting QAT’s gradient-based backtracking mechanism, we examine how quantization parameters affect oscillation. Inspired by [30], we observe that modifying quantization thresholds can induce more weights to oscillate around them, as shown in Figure 3. Specifically, when the intervals between quantization thresholds and the differences among quantization levels decrease, weights tend to transition from intra-bin oscillation to threshold-adjacent oscillation. This behavior is driven by two factors. First, the shifting thresholds force certain weights to cross them, introducing noise into the network. The facilitating effect of smaller quantization threshold intervals on weight oscillation will be demonstrated in Section 3.4. Second, the reduced differences between quantization levels lower the penalty for mapping errors when floating-point weights are mapped to quantized values, making weights more likely to oscillate near the thresholds, as smaller differences between quantization levels tend to smooth the loss surface.

To leverage the weight oscillation mentioned above, we decouple the quantization thresholds and levels and introduce a three-stage training strategy. Specifically, we manually adjust the quantization thresholds to gradually reduce their intervals, while setting the quantization levels as learnable parameters to optimize during training. Additionally, we introduce differentiability for clipped weights in fake quantization to address the issue observed in Section 3.2, where the absence of gradients in clipped weights (as shown in Eq. (2)) hinders model fine-tuning. In the final stage, oscillation is suppressed to stabilize convergence. The complete strategy is outlined below:

- **Stage 1: Inducing oscillation.** For weight quantization,

we define the intervals between thresholds as s_{th} and between quantization levels as s_{le} , both initialized to $\frac{\sigma_w}{2^{b-2}}$, where σ_w is the standard deviation of weights in a layer, and b is the bit-width. In this stage, we manually reduce s_{th} and s_{le} to promote oscillation at thresholds.

- **Stage 2: Learning quantization levels.** We fix s_{th} and set s_{le} as a learnable parameter for optimization. In Stage 1, s_{le} is manually reduced, which leads to higher loss due to poor representation of weights by the quantization levels. As training progresses and the task loss decreases, s_{le} is learned to increase automatically. This widens the intervals between quantization levels, makes the loss landscape steeper near thresholds, and reduces oscillation by discouraging weights from frequently crossing thresholds. The quantizer is defined as:

$$\hat{w} = \begin{cases} \text{level}[0] & \text{if } w < \text{thresh}[0] \\ \text{level}[i] & \text{if } \text{thresh}[i-1] \leq w < \text{thresh}[i] \\ \text{level}[2^b-1] & \text{if } w \geq \text{thresh}[2^b-2] \end{cases} \quad (6)$$

where $i \in \{1, 2, \dots, 2^b-2\}$, and $\text{thresh}[i]$ and $\text{level}[i]$ represent the (i+1)-th threshold and level, respectively. The optimized quantized weights are denoted as:

$$\tilde{w} = \hat{w} + w - w. \text{detach}(\cdot). \quad (7)$$

This formulation, based on the STE, enables distinct gradient flows: $\hat{w}$ determines the gradient for s_{le} , while w receives gradients directly from the task loss, even when clipped. This decoupling avoids the zero-gradient issue and enables more effective optimization of both weights and quantizer parameters.

- **Stage 3: Dampening oscillation.** In the third stage, we employ the *Oscillation Dampening* method outlined in [36], which adds a regularization term to the loss:

$$\mathcal{L} = \mathcal{L}_{\text{task}} + \lambda \mathcal{L}_{\text{dampen}}, \quad (8)$$

where λ is a hyperparameter. The dampening loss is:

$$\mathcal{L}_{\text{dampen}} = \|\hat{w} - \text{clip}(w, Q_n, Q_p)\|_F^2, \quad (9)$$

which penalizes weights lingering near quantization thresholds, encouraging them to converge to the nearest quantization centers.

Details regarding the hyperparameters and transition schedule for each training stage are provided in Section 4.1.

3.4. Theoretical Analysis

In this section, we theoretically analyze the effectiveness of our approach and compare it against prior differentiable quantization parameter methods.

We first show the advantage of using smaller quantization threshold intervals. Let W be the set of pre-trained

weights, and Q be a set composed of quantized values. Weight quantization is a mapping from the W to the Q . For a given weight $w \in W$, we define its probability of being quantized to $Q_i \in Q$ as the transition probability $P(Q_i | w)$, where $i \in \{0, 1, \dots, 2^b - 1\}$ and b is the bit-width. By Bayes' theorem:

$$P(Q_i | w) = \frac{P(Q_i) \cdot P(w | Q_i)}{P(w)}, \quad (10)$$

where $P(Q_i)$ is the proportion of the number of weights mapped to Q_i to the total number of weights, $P(w)$ is the probability distribution of weights, and $P(w | Q_i)$ is the conditional distribution of weights mapped to Q_i . The total probability formula applied to Eq. (10) is:

$$P(Q_i | w) = \frac{P(Q_i) \cdot P(w | Q_i)}{\sum_i P(Q_i) \cdot P(w | Q_i)}. \quad (11)$$

As shown in Figure 2a, $P(w | Q_i)$ approximately follows a Gaussian distribution, so the total probability formula can be transformed into a Gaussian mixture model:

$$P(w) = \sum_i \pi_i \mathcal{N}(w | \mu_i, \sigma_i), \quad (12)$$

where π_i, μ_i, σ_i denote the weight, mean and variance of the Gaussian component, respectively. This transformation does not effect the theoretical conclusion, just for analytical convenience. Eq. (11) is transformed into:

$$P(Q_i | w) = \frac{\pi_i \mathcal{N}(w | \mu_i, \sigma_i)}{\sum_i \pi_i \mathcal{N}(w | \mu_i, \sigma_i)}. \quad (13)$$

The weight w is assigned to the quantization value Q_j that maximizes this probability:

$$\begin{aligned} \text{if } \forall i, \pi_j \mathcal{N}(w | \mu_j, \sigma_j) &\geq \pi_i \mathcal{N}(w | \mu_i, \sigma_i) \\ \text{then } w &\rightarrow Q_j. \end{aligned} \quad (14)$$

Therefore, when the quantization threshold is set at the horizontal coordinate value of the intersection between two adjacent Gaussian components, it ensures optimal probabilistic mapping of weights. In Figure 2a, we observe that in the lossless solution for this example, the variances of each Gaussian component are approximately equal, denoted as σ . Thus, the horizontal coordinate value at the intersection of adjacent Gaussian components is:

$$\text{thresh}[i] = \frac{\mu_{i+1} + \mu_i}{2} - \frac{\log\left(\frac{\pi_{i+1}}{\pi_i}\right) \cdot \sigma^2}{\mu_{i+1} - \mu_i}. \quad (15)$$

This analysis suggests that, when Gaussian component means are close, using smaller threshold intervals improves mapping accuracy. It explains why our method encourages

tighter thresholds in early training to support more effective weight assignment in QAT.

Next, we show that AOQ provides a more direct and accurate link between quantization levels and task loss than previous approaches. In prior methods [5, 12, 49], learnable quantization parameters jointly determine both thresholds and levels. This coupling introduces approximation errors in the gradient computation of quantization levels. For instance, in LSQ [12], the gradients of the scaling factors are shown in Eq. (3). The derivative of weights with respect to the scaling factor aids in optimizing quantization thresholds but makes the gradient of the quantization levels inaccurate. In contrast, AOQ decouples thresholds from levels. It directly optimizes quantization levels based on Eq. (6) and Eq. (7), allowing a more faithful gradient flow that better aligns with task loss.

Finally, AOQ effectively addresses the impact of weight oscillation on final convergence, as well as on batch normalization and layer normalization. After exploring a broader solution space, we use *Oscillation Dampening* in the final stage to gradually guide weights out of oscillatory regions. As training continues, the exponential moving average used in batch normalization and layer normalization becomes increasingly stable, reinforcing consistent model behavior and improving convergence quality.

4. Experiments

We evaluate the effectiveness of the proposed Allowing Oscillation Quantization (AOQ) on the ILSVRC12 ImageNet dataset [9], using a range of models including ResNet [15], MobileNet-V2 [17], DeiT [41], and Swin Transformer [33]. Our experiments focus on low-bit weight quantization. For activation quantization, we adopt LSQ [12] and N2UQ [32].

4.1. Experimental Setup

ResNet and MobileNet We follow the training schemes in [5, 12, 32], initializing quantized networks from full-precision pre-trained models. Training is performed using the Adam optimizer [20] with a linear learning rate decay, starting from 1×10^{-3} . Following LSQ [12], we apply knowledge distillation and train for 150 epochs. All convolutional and fully connected layers are quantized, except for the first and last layers.

DeiT and Swin Transformer We use the official implementations and adopt training settings from [28–30], including knowledge distillation and a training duration of 300 epochs. As in prior work, the patch embedding and classification layers are quantized to 8 bits [28–30].

Parameter Setup The batch size is set to 512, and the weight decay is set to 0. The three training stages occupy 1/5, 2/5, and 2/5 of the total training epochs, respectively. We reduce s_{th} and s_{le} to 30%–35% of their initial values in the first stage. As shown in Fig. 4, a broad grid search

Table 2. Accuracy comparison to the state-of-the-art quantization methods with ResNet structure on ImageNet dataset. Note that W/A denote the bit-widths of weights and activations, respectively. FP denotes the top-1 accuracy of the full-precision models.

Network	Method	Bit-width (W/A)	Accuracy (%) Top-1	Bit-width (W/A)	Accuracy(%) Top-1	Bit-width (W/A)	Accuracy(%) Top-1
ResNet-18 (FP: 71.8)	LQ-Nets [46]	2/2	64.9	3/3	68.2	4/4	69.3
	DSQ [13]	2/2	65.2	3/3	68.7	4/4	69.6
	QIL [19]	2/2	65.7	3/3	69.2	4/4	70.1
	APoT [27]	2/2	67.3	3/3	69.9	4/4	70.7
	LSQ [12]	2/2	67.6	3/3	70.2	4/4	71.1
	LCQ [43]	2/2	68.9	3/3	70.6	4/4	71.5
	N2UQ [32]	2/2	69.4	3/3	71.9	4/4	72.9
	LSQ+AOQ	2/2	69.7	3/3	71.4	4/4	72.0
ResNet-34 (FP:74.9)	LQ-Nets [46]	2/2	69.8	3/3	71.9	4/4	-
	DSQ [13]	2/2	70.0	3/3	72.5	4/4	72.8
	QIL [19]	2/2	70.6	3/3	73.1	4/4	73.7
	APoT [27]	2/2	70.9	3/3	73.4	4/4	73.8
	LSQ [12]	2/2	71.6	3/3	73.4	4/4	74.1
	LCQ [43]	2/2	72.7	3/3	74.0	4/4	74.3
	N2UQ [32]	2/2	73.3	3/3	75.2	4/4	76.0
	LSQ+AOQ	2/2	73.5	3/3	74.9	4/4	75.5
ResNet-50 (FP: 77.0)	LQ-Nets [46]	2/2	71.5	3/3	74.2	4/4	75.1
	APoT [27]	2/2	73.4	3/3	75.8	4/4	76.6
	LSQ [12]	2/2	73.7	3/3	75.8	4/4	76.7
	LCQ [43]	2/2	75.1	3/3	76.3	4/4	76.6
	N2UQ [32]	2/2	75.8	3/3	77.5	4/4	78.0
	LSQ+AOQ	2/2	76.3	3/3	77.5	4/4	77.9
	N2UQ+AOQ	2/2	76.4	3/3	77.8	4/4	78.1

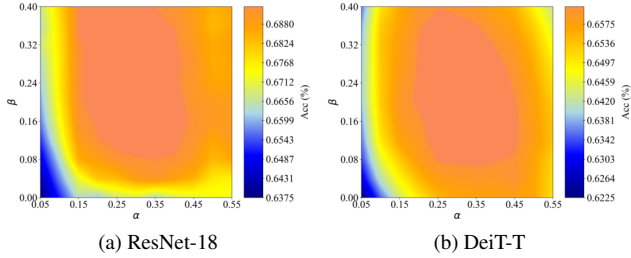


Figure 4. Accuracy heatmaps on ImageNet under different narrowing schedules. α denotes the final-to-initial ratio of s_{le} and s_{th} in stage one, and β is the proportion of total epochs allocated to this stage. AOQ maintains stable performance across a wide range of (α, β) settings on both (a) 2-bit ResNet-18 and (b) 2-bit DeiT-T, indicating schedule robustness across architectures.

confirms that a common schedule performs robustly across different architectures in practice. For the final stage, the hyper-parameter λ is chosen such that $\lambda \mathcal{L}_{\text{dampen}}$ is two to three orders of magnitude lower than $\mathcal{L}_{\text{task}}$.

4.2. Experimental Results on Toy Example

We first revisit the toy example from Section 3.2 to evaluate the effect of oscillation on quantization quality. To

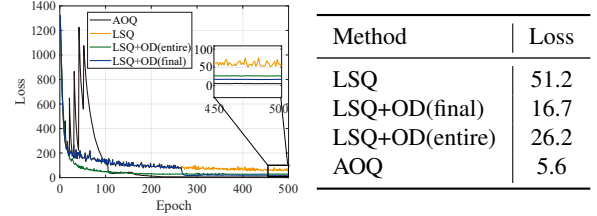


Figure 5. Loss curves and minimum loss obtained from solving the toy example using various methods.

demonstrate that weight oscillation can expand the solution space and improve quantization outcomes, we apply the *Oscillation Dampening* (OD) [36] to suppress oscillation either during the final training stage or throughout the entire training process. We also evaluate the performance of AOQ using the toy example. As shown in Figure 5, allowing early-stage oscillation improves QAT performance, and AOQ converges faster to a better solution. When using fixed quantization levels $\{-2, -1, 0, 1\}$ in the third stage, Figure 2c shows that AOQ enables pre-trained weights to be mapped to more diverse and distant quantized values, expanding the solution space compared to Figure 2b.

Table 3. Top-1 accuracy (%) comparison for methods suppressing weight oscillation throughout the process in MobileNet-V2.

Network	Method	W3A3	W4A4
MobileNet-V2 (FP: 72.1)	LSQ [12]	65.3	69.5
	LSQ+BR [14]	67.4	70.4
	LSQ+Dampen [36]	67.8	70.5
	N2UQ [32]	69.0	72.1
	LSQ+AOQ	69.1	71.8
	N2UQ+AOQ	69.9	72.5

Table 4. Top-1 accuracy (%) comparison with the state-of-the-art methods in the ViT architecture.

Network	Method	W2A2	W3A3
DeiT-T (FP: 72.1)	LSQ [12]	54.5	68.1
	QViT* [28]	50.4	67.1
	OFQ [30]	64.3	72.7
	LSQ+AOQ	66.5	72.3
DeiT-S (FP: 79.9)	LSQ [12]	68.0	77.8
	QViT* [28]	68.7	78.5
	OFQ [30]	75.7	79.6
	LSQ+AOQ	76.7	79.8
Swin-T (FP: 81.2)	LSQ [12]	70.4	79.0
	QViT* [28]	73.9	80.1
	OFQ [30]	78.5	81.1
	LSQ+AOQ	78.9	81.2

4.3. Accuracy Evaluation

Table 2 reports the accuracy of AOQ on ResNet. When paired with different activation quantization techniques, AOQ consistently improves performance. For instance, using AOQ with LSQ yields 0.9%–2.6% accuracy gains. When combined with N2UQ, AOQ outperforms many traditional methods. Its advantage becomes more pronounced in low-bit networks with a large number of weights, where the expanded solution space is more effectively utilized. In the 2-bit ResNet-50 case, AOQ reduces the accuracy gap with full-precision to just 0.6%.

Table 3 shows the performance of the AOQ method on MobileNet-V2. Compared to other methods, AOQ has a natural advantage in quantizing MobileNet. Due to significant variations in weight distributions across channels in the depth-wise separable convolution layers, traditional quantization methods often map all weights in certain channels to a single quantized value, resulting in the loss of important information. AOQ mitigates this by adaptively shrinking threshold intervals during training, promoting threshold-crossing behavior and improving quantization granularity. This effect is illustrated in Figure 3. Overall, AOQ outper-

Table 5. Effects of different components of our strategy on the final performance.

Method	Acc (%)
Baseline [12]	73.7
– Weight quantization	76.6
+ Expanding solution space of weights	75.2
+ Delayed weight oscillation suppression	75.6
+ Both	76.3
Corresponding real-valued network	77.0

forms other QAT approaches, including those that suppress oscillation throughout [14, 36].

To validate AOQ on Vision Transformers (ViT), we conducted 2-bit and 3-bit quantization experiments on DeiT and Swin Transformer. We observe that QViT [28] does not fully quantize activations in the fully connected and attention layers, limiting hardware acceleration. For fair comparison, we modify their implementation accordingly and report the results as QViT* in Table 4. The results show that AOQ significantly outperforms LSQ and QViT by suppressing weight oscillation, boosting 2-bit DeiT-T accuracy by 12.0% and 16.1%, respectively. By expanding the solution space, AOQ also surpasses OFQ, the prior state-of-the-art, under 2-bit quantization.

4.4. Ablation Study

To investigate the impact of each improvement on the final performance, we conduct an ablation study on the 2-bit quantized ResNet-50 network on ImageNet. As shown in Table 5, the proposed expanded solution space and delayed weight oscillation suppression improve the accuracy by 1.5% and 1.9%, respectively over the baseline [12]. By using both techniques, we reduced the accuracy gap with the corresponding real-valued ResNet-50 to only 0.7%, and the accuracy is only reduced by 0.3% compared to the baseline method without weight quantization.

5. Conclusion

In this work, we reveal the weakness of existing QAT methods in exploring the quantization solution space of weights within low bit-width constraints, which adversely impacts the performance of quantized networks. We highlight the importance of weight oscillation during early training, as it helps the optimizer escape local minima and explore a broader set of quantized configurations. Based on this insight, we propose a simple yet effective method that encourages weight oscillation in the early stage to expand the solution space, followed by oscillation suppression to stabilize convergence. Experiments demonstrate that our algorithm quantizes various models to low-bits and achieves state-of-the-art accuracy on ImageNet.

Acknowledgments

This work was supported in part by National Natural Science Foundation of China under Grant 62322117, 62425109, 62371365, U24B20136, and U22B2014.

References

- [1] Haoli Bai, Wei Zhang, Lu Hou, Lifeng Shang, Jing Jin, Xin Jiang, Qun Liu, Michael R. Lyu, and Irwin King. Binarybert: Pushing the limit of bert quantization. In *Annual Meeting of the Association for Computational Linguistics*, 2020. 1
- [2] Yoshua Bengio, Nicholas Léonard, and Aaron Courville. Estimating or propagating gradients through stochastic neurons for conditional computation. *arXiv preprint arXiv:1308.3432*, 2013. 3
- [3] Hanlin Chen, Li'an Zhuo, Baochang Zhang, Xiwu Zheng, Jianzhuang Liu, Rongrong Ji, David S. Doermann, and Guodong Guo. Binarized neural architecture search for efficient object recognition. *International Journal of Computer Vision*, 129:501–516, 2020. 1
- [4] Shangyu Chen, Wenya Wang, and Sinno Jialin Pan. Metaquant: Learning to quantize by learning to penetrate non-differentiable quantization. In *Neural Information Processing Systems*, 2019. 2
- [5] Jungwook Choi, Zhuo Wang, Swagath Venkataramani, Pierce I-Jen Chuang, Vijayalakshmi Srinivasan, and Kailash Gopalakrishnan. Pact: Parameterized clipping activation for quantized neural networks. *arXiv preprint arXiv:1805.06085*, 2018. 2, 6
- [6] Michele Conforti, Gérard Cornuéjols, Giacomo Zambelli, Michele Conforti, Gérard Cornuéjols, and Giacomo Zambelli. *Integer programming models*. Springer, 2014. 3
- [7] Matthieu Courbariaux, Yoshua Bengio, and Jean-Pierre David. Binaryconnect: Training deep neural networks with binary weights during propagations. In *Neural Information Processing Systems*, 2015. 2
- [8] Alexandre Défossez, Yossi Adi, and Gabriel Synnaeve. Differentiable model compression via pseudo quantization noise. *Transactions on Machine Learning Research*, 2021. 1, 2
- [9] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 248–255. Ieee, 2009. 3, 6
- [10] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. Gpt3. int8 (): 8-bit matrix multiplication for transformers at scale. In *Neural Information Processing Systems*, pages 30318–30332, 2022. 1
- [11] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. Qlora: Efficient finetuning of quantized llms. In *Neural Information Processing Systems*, 2024. 1
- [12] Steven K Esser, Jeffrey L McKinstry, Deepika Bablani, Rathinakumar Appuswamy, and Dharmendra S Modha. Learned step size quantization. *arXiv preprint arXiv:1902.08153*, 2019. 1, 2, 3, 4, 6, 7, 8
- [13] Ruihao Gong, Xianglong Liu, Shenghu Jiang, Tian-Hao Li, Peng Hu, Jiazhen Lin, Fengwei Yu, and Junjie Yan. Differentiable soft quantization: Bridging full-precision and low-bit neural networks. In *IEEE/CVF International Conference on Computer Vision*, pages 4851–4860, 2019. 1, 2, 7
- [14] Tiantian Han, Dong Li, Ji Liu, Lu Tian, and Yi Shan. Improving low-precision network quantization via bin regularization. In *IEEE/CVF International Conference on Computer Vision*, pages 5241–5250, 2021. 8
- [15] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 770–778, 2016. 6
- [16] Yefei He, Luping Liu, Jing Liu, Weijia Wu, Hong Zhou, and Bohan Zhuang. Ptdq: Accurate post-training quantization for diffusion models. In *Neural Information Processing Systems*, 2024. 1
- [17] Andrew G Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, and Hartwig Adam. Mobilenets: Efficient convolutional neural networks for mobile vision applications. *arXiv preprint arXiv:1704.04861*, 2017. 6
- [18] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew G. Howard, Hartwig Adam, and Dmitry Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 2704–2713, 2017. 1, 2
- [19] San Hun Jung, Changyong Son, Seohyung Lee, JinWoo Son, Jae-Joon Han, Youngjun Kwak, Sung Ju Hwang, and Changkyu Choi. Learning to quantize deep networks by optimizing quantization intervals with task loss. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 4345–4354, 2018. 2, 7
- [20] Diederik P Kingma. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014. 6
- [21] Scott Kirkpatrick, Charles D. Gelatt, and Mario P. Vecchi. Optimization by simulated annealing. *Science*, 220:671–680, 1983. 5
- [22] Yann LeCun, Léon Bottou, Genevieve B Orr, and Klaus-Robert Müller. Efficient backprop. In *Neural networks: Tricks of the trade*, pages 9–50. Springer, 2002. 2
- [23] Junghyup Lee, Dohyung Kim, and Bumsu Ham. Network quantization with element-wise gradient scaling. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 6444–6453, 2021. 1, 2
- [24] Jung Hyun Lee, Jeonghoon Kim, Se Jung Kwon, and Dongsoo Lee. Flexround: Learnable rounding based on element-wise division for post-training quantization. In *International Conference on Machine Learning*, 2023. 1
- [25] Cong Leng, Hao Li, Shenghuo Zhu, and Rong Jin. Extremely low bit neural network: Squeeze the last bit out with admm. In *AAAI Conference on Artificial Intelligence*, 2017. 2
- [26] Xinlin Li, Bang Liu, Ruizhi Yang, Vanessa Courville, Chao Xing, and V. Nia. Denseshift: Towards accurate and efficient low-bit power-of-two quantization. In *IEEE/CVF International Conference on Computer Vision*, pages 16964–16974, 2022. 2
- [27] Yuhang Li, Xin Dong, and Wei Wang. Additive powers-of-two quantization: An efficient non-uniform discretization for

- neural networks. In *International Conference on Learning Representations*, 2019. 1, 2, 7
- [28] Yanjing Li, Sheng Xu, Baochang Zhang, Xianbin Cao, Peng Gao, and Guodong Guo. Q-vit: Accurate and fully quantized low-bit vision transformer. In *Neural Information Processing Systems*, pages 34451–34463, 2022. 6, 8
- [29] Zhixin Li, Tong Yang, Peisong Wang, and Jian Cheng. Q-vit: Fully differentiable quantization for vision transformer. *arXiv preprint arXiv:2201.07703*, 2022.
- [30] Shi Liu, Zechun Liu, and Kwang-Ting Cheng. Oscillation-free quantization for low-bit vision transformers. In *International Conference on Machine Learning*, 2023. 1, 2, 4, 5, 6, 8
- [31] Zechun Liu, Baoyuan Wu, Wenhan Luo, Xin Yang, Wei Liu, and Kwang-Ting Cheng. Bi-real net: Enhancing the performance of 1-bit cnns with improved representational capability and advanced training algorithm. In *European Conference on Computer Vision*, pages 722–737, 2018. 1
- [32] Zechun Liu, Kwang-Ting Cheng, Dong Huang, Eric P. Xing, and Zhiqiang Shen. Nonuniform-to-uniform quantization: Towards accurate quantization via generalized straight-through estimation. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 4932–4942, 2021. 1, 2, 6, 7, 8
- [33] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. Swin transformer: Hierarchical vision transformer using shifted windows. In *IEEE/CVF International Conference on Computer Vision*, pages 9992–10002, 2021. 6
- [34] Ilya Loshchilov and Frank Hutter. Sgdr: Stochastic gradient descent with warm restarts. *arXiv preprint arXiv:1608.03983*, 2016. 3
- [35] Markus Nagel, Rana Ali Amjad, Mart Van Baalen, Christos Louizos, and Tijmen Blankevoort. Up or down? adaptive rounding for post-training quantization. In *International Conference on Machine Learning*, pages 7197–7206. PMLR, 2020. 1
- [36] Markus Nagel, Marios Fournarakis, Yelysei Bondarenko, and Tijmen Blankevoort. Overcoming oscillations in quantization-aware training. In *International Conference on Machine Learning*, 2022. 1, 2, 4, 5, 7, 8
- [37] Yury Nahshan, Brian Chmiel, Chaim Baskin, Evgenii Zheltonozhskii, Ron Banner, Alexander M. Bronstein, and Avi Mendelson. Loss aware post-training quantization. *Machine Learning*, 110:3245 – 3262, 2019. 2, 3
- [38] Juncheol Shin, Junhyuk So, Sein Park, Seungyeop Kang, Sungjoo Yoo, and Eunhyeok Park. Nipq: Noise proxy-based integrated pseudo-quantization. *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 3852–3861, 2022. 2
- [39] Zhenhong Sun, Ce Ge, Junyan Wang, Ming Lin, Hesun Chen, Hao Li, and Xiuyu Sun. Entropy-driven mixed-precision quantization for deep network design. In *Neural Information Processing Systems*, 2022. 1
- [40] Charles E. Taylor. Adaptation in natural and artificial systems: An introductory analysis with applications to biology, control, and artificial intelligence. *The Quarterly Review of Biology*, 69:88–89, 1994. 5
- [41] Hugo Touvron, Matthieu Cord, Matthijs Douze, Francisco Massa, Alexandre Sablayrolles, and Hervé Jégou. Training data-efficient image transformers & distillation through attention. In *International Conference on Machine Learning*, 2020. 6
- [42] Ziwei Wang, Ziyi Wu, Jiwen Lu, and Jie Zhou. Bidet: An efficient binarized object detector. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 2046–2055, 2020. 1
- [43] Kohei Yamamoto. Learnable companding quantization for accurate low-bit neural networks. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 5027–5036, 2021. 2, 7
- [44] Jiwei Yang, Xu Shen, Jun Xing, Xinmei Tian, Houqiang Li, Bing Deng, Jianqiang Huang, and Xian-sheng Hua. Quantization networks. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 7308–7316, 2019. 2
- [45] Penghang Yin, Jiancheng Lyu, Shuai Zhang, Stanley Osher, Yingyong Qi, and Jack Xin. Understanding straight-through estimator in training activation quantized neural nets. In *International Conference on Learning Representations*, 2019. 1, 2
- [46] Dongqing Zhang, Jiaolong Yang, Dongqiangzi Ye, and Gang Hua. Lq-nets: Learned quantization for highly accurate and compact deep neural networks. In *European Conference on Computer Vision*, 2018. 1, 2, 7
- [47] Jiayi Zhang, Chang Liu, Junchi Yan, Xijun Li, Hui-Ling Zhen, and Min jie Yuan. A survey for solving mixed integer programming via machine learning. *Neurocomputing*, 519:205–217, 2022. 3
- [48] Yifan Zhang, Zhen Dong, Huanrui Yang, Ming Lu, Cheng-Ching Tseng, Yuan Du, Kurt Keutzer, Li Du, and Shanghang Zhang. Qd-bev : Quantization-aware view-guided distillation for multi-view 3d object detection. In *IEEE/CVF International Conference on Computer Vision*, pages 3802–3812, 2023. 1
- [49] Shuchang Zhou, Yuxin Wu, Zekun Ni, Xinyu Zhou, He Wen, and Yuheng Zou. Dorefa-net: Training low bitwidth convolutional neural networks with low bitwidth gradients. *arXiv preprint arXiv:1606.06160*, 2016. 1, 2, 6